

CSCI 5380 - Network Virtualization and Orchestration

Lab 8 BGP SDN

University of Colorado Boulder
Department of Computer Science
Network Engineering

Group Assignment:
Sarang, Pranav, Aravindh, Atul, Zeqian

Summary:

In this Lab, you will configure BGP in a virtualized, traditional, and SDN. See Figure 1.

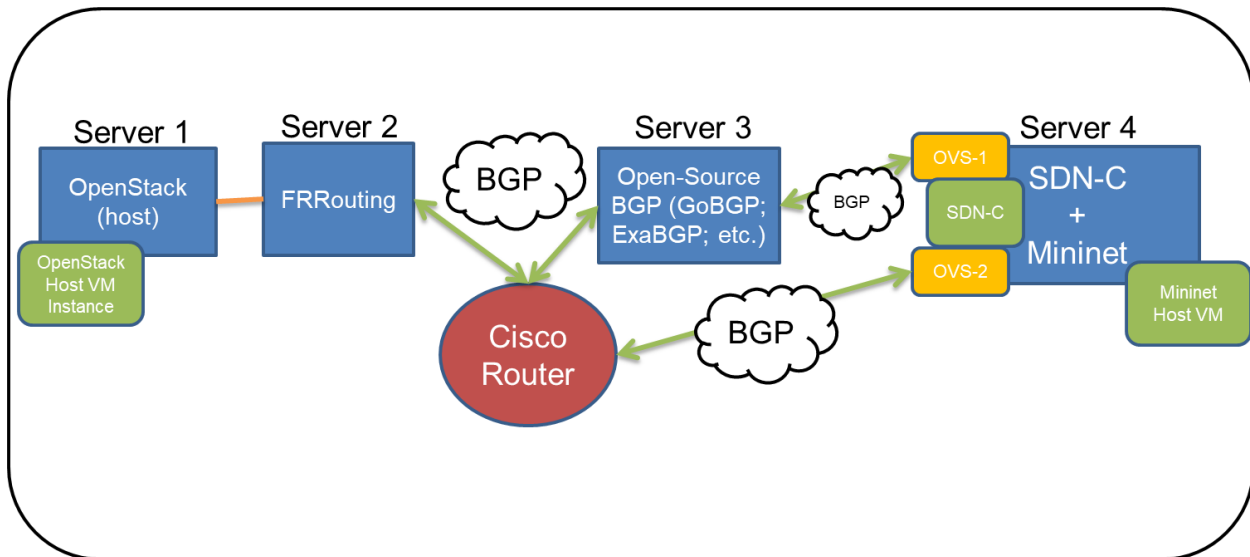


Figure 1: Lab environment

Each team will use one Cisco router and four servers running-

1. OpenStack
2. FRRouting [<https://frrouting.org/>]
3. SDN Controller (Ryu) + Mininet
4. Open-Source BGP (your choice)

Section 1 - Establish BGP Peering between FRRouting and Cisco router:

Configure BGP on FRRouting server and establish peering with the Cisco router.

[FRRouting is forked from Quagga and the installation process is the same. You can refer this document and replace Quagga with FRRouting -

Step 1: Configure the IP on Interface connecting Server 2 to Cisco Router

```
root@frrouting:/etc/frr# ip link set eno1 up
root@frrouting:/etc/frr# ip addr add 10.25.1.1/24 dev eno1
```

Step 2: Enable BGP on FRR

```
root@frrouting:/etc/frr# cat daemons | grep bgp
bgpd=yes
bgpd_options="-A 127.0.0.1"
# bgpd_wrap="/usr/bin/daemonize /usr/bin/mywrapper"
```

Step 3: Add Configuration to frr.conf

```
root@frrouting:/etc/frr# cat frr.conf
# default to using syslog. /etc/rsyslog.d/45-frr.conf places the log in
# /var/log/frr/frr.log
#
# Note:
# FRR's configuration shell, vtysh, dynamically edits the live, in-memory
# configuration while FRR is running. When instructed, vtysh will persist the
# live configuration to this file, overwriting its contents. If you want to
# avoid this, you can edit this file manually before starting FRR, or instruct
# vtysh to write configuration to a different file.

frr version 8.4
frr defaults traditional
!
router bgp 200
  bgp router-id 2.2.2.2
  neighbor 10.25.1.2 remote-as 300
  neighbor 10.25.1.2 description Cisco-Router
  !
  address-family ipv4 unicast
    neighbor 10.25.1.2 activate
  exit-address-family
!
line vty
!
```

Step 4: Restart the Process

```
root@frrouting:/etc/frr# systemctl status frr
● frr.service - FRRouting
   Loaded: loaded (/usr/lib/systemd/system/frr.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-03-11 03:41:06 UTC; 19min ago
     Docs: https://frrouting.readthedocs.io/en/latest/setup.html
   Process: 5646 ExecStart=/usr/lib/frr/frrinit.sh start (code=exited, status=0/SUCCESS)
  Main PID: 5658 (watchfrr)
    Status: "FRR Operational"
```

Step 5: Check Neighborhood before Configuring Router

```
root@frrouting:/etc/frr# sudo vtysh -c "show bgp summary"

IPv4 Unicast Summary (VRF default):
BGP router identifier 2.2.2.2, local AS number 200 vrf-id 0
BGP table version 0
RIB entries 0, using 0 bytes of memory
Peers 1, using 724 KiB of memory

Neighbor      V      AS  MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down  State/PfxRcd  PfxSnt  Desc
10.25.1.2     4       300       0        0        0    0    0    never     Active         0  Cisco-Router
```

Step 6: Configure Cisco Router's Interface

```

interface GigabitEthernet0/0
  description connect to management switch
  ip address 10.0.0.15 255.255.255.0
  duplex auto
  speed auto
  media-type rj45

```

Step 7: Configure BGP on Cisco Router

```

router bgp 300
  bgp router-id 3.3.3.3
  bgp log-neighbor-changes
  neighbor 10.25.1.1 remote-as 200
  neighbor 10.25.1.1 description FRR-Ubuntu
  !
  address-family ipv4
    neighbor 10.25.1.1 activate
  exit-address-family

```

Step 8: Verify BGP Neighborhood on both devices

```

Cisco_Router#show ip bgp summary
BGP router identifier 3.3.3.3, local AS number 300
BGP table version is 1, main routing table version 1

Neighbor        V    AS MsgRcvd MsgSent  TblVer  InQ OutQ Up/Down  State/PfxRcd
10.25.1.1        4    200   179    201      1     0   0 02:57:52      0

```

```

lab@frrouting:~$ sudo vtysh -c "show bgp summary"
[sudo] password for lab:

IPv4 Unicast Summary (VRF default):
BGP router identifier 2.2.2.2, local AS number 200 vrf-id 0
BGP table version 0
RIB entries 0, using 0 bytes of memory
Peers 1, using 724 KiB of memory

Neighbor        V    AS MsgRcvd MsgSent  TblVer  InQ OutQ Up/Down  State/PfxRcd  PfxSnt Desc
10.25.1.2        4    300   202    180      0     0   0 02:58:56      (Policy) (Policy) Cisco-Router

Total number of neighbors 1
lab@frrouting:~$

```

Section 2 - Establish BGP Peering between Cisco Router and Open-Source BGP speaker: ?

Configure BGP on Cisco Router and establish a peering relationship with the Open-Source BGP speaker of your choice (GoBGP or ExaBGP recommended).

Exabgp was used to complete this objective.

1. Installing ExaBGP:

Sudo pip3 install exabgp

```
lab@bgp-ser:~$ sudo pip3 install exabgp
```

2. Create the necessary exabgp directories and conf file:

sudo mkdir -p /etc/exabgp

sudo vi /etc/exabgp/exabgp.conf

```
lab@bgp-ser:/etc$ sudo mkdir -p /etc/exabgp
lab@bgp-ser:/etc$ sudo vi /etc/exabgp/exabgp.conf
```

Exabgp.conf file config

```
process relay {
    run /etc/exabgp/relay.py;
    encoder json;
}

neighbor 10.35.1.2 {
    description "R1 AS300";
    router-id 10.35.1.1;
    local-address 10.35.1.1;
    local-as 400;
    peer-as 300;

    family {
        ipv4 unicast;
    }
}

neighbor 10.45.1.1 {
    description "R2 AS500";
    router-id 10.45.1.2;
    local-address 10.45.1.2;
    local-as 400;
    peer-as 500;

    family {
        ipv4 unicast;
    }
}
```

3. Now create the python script relay.py

```
#!/usr/bin/env python3
import sys
import json

sys.stdout.flush()

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue

    try:
        msg = json.loads(line)

        # Only care about BGP update messages
        if msg.get("type") != "update":
            continue

        neighbor = msg.get("neighbor", {})
        peer_ip = neighbor.get("address", {}).get("peer", "")
        update = neighbor.get("message", {}).get("update", {})

        # — Announcements —————
        announce_block = update.get("announce", {}).get("ipv4 unicast", {})
        for nexthop, prefixes in announce_block.items():
            for prefix in prefixes:
                # next-hop self so the receiving router can resolve it
                sys.stdout.write(f"announce route {prefix} next-hop self\n")
                sys.stdout.flush()

        # — Withdrawals —————
        withdraw_block = update.get("withdraw", {}).get("ipv4 unicast", [])
        for prefix in withdraw_block:
            sys.stdout.write(f"withdraw route {prefix}\n")
            sys.stdout.flush()

    except (json.JSONDecodeError, KeyError, TypeError):

except (json.JSONDecodeError, KeyError, TypeError):
    pass
```

Commands to sort out permission issues:

```
lab@bgp-ser:/etc$ sudo rm /run/exabgp/exabgp.in
sudo rm /run/exabgp/exabgp.out
sudo mkfifo /run/exabgp/exabgp.in
sudo mkfifo /run/exabgp/exabgp.out
sudo chmod 666 /run/exabgp/exabgp.in
sudo chmod 666 /run/exabgp/exabgp.out
```

4. For exabgp to be persistent. We created a service for it.

```
lab@bgp-ser:/etc$ sudo vi /etc/systemd/system/exabgp.service
lab@bgp-ser:/etc$ sudo systemctl daemon-reload
```

```
[Unit]
Description=ExaBGP BGP Route Server
After=network.target
Wants=network.target

[Service]
Type=simple
User=root
ExecStartPre=/bin/mkdir -p /run/exabgp
ExecStartPre=/bin/bash -c 'rm -f /run/exabgp/exabgp.in /run/exabgp/exabgp.out && mkfifo /run/exabgp/exabgp.in /run/exabgp/exabgp.out && chmod 666 /run/exabgp/exabgp.in /run/exabgp/exabgp.out'
ExecStart=/usr/local/bin/exabgp server /etc/exabgp/exabgp.conf
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
~
~
```

```
lab@bgp-ser:/etc$ sudo systemctl daemon-reload
lab@bgp-ser:/etc$ sudo s
Display all 258 possibilities? (y or n)
lab@bgp-ser:/etc$ sudo systemctl enable exabgp
Created symlink /etc/systemd/system/multi-user.target.wants/exabgp.service → /etc/systemd/system/exabgp.service.
lab@bgp-ser:/etc$ sudo systemctl start exabgp
Failed to start exabgp.service: Unit exabgp.service not found.
lab@bgp-ser:/etc$ sudo systemctl start exabgp
lab@bgp-ser:/etc$ sudo systemctl status exabgp
● exabgp.service - ExaBGP BGP Route Server
   Loaded: loaded (/etc/systemd/system/exabgp.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-03-17 02:41:45 UTC; 5s ago
     Process: 18766 ExecStartPre=/bin/mkdir -p /run/exabgp (code=exited, status=0/SUCCESS)
     Process: 18768 ExecStartPre=/bin/bash -c rm -f /run/exabgp/exabgp.in /run/exabgp/exabgp.out && mkfifo /run/exabgp (code=exited, status=0/SUCCESS)
    Main PID: 18772 (exabgp)
      Tasks: 3 (limit: 77059)
     Memory: 40.9M (peak: 41.5M)
        CPU: 760ms
    CGroup: /system.slice/exabgp.service
            └─18772 /usr/bin/python3 /usr/local/bin/exabgp server /etc/exabgp/exabgp.conf
              └─18775 /usr/bin/python3 /usr/local/bin/exabgp
                └─18776 python3 /etc/exabgp/relay.py

Mar 17 02:41:45 bgp-ser systemd[1]: Starting exabgp.service - ExaBGP BGP Route Server...
Mar 17 02:41:45 bgp-ser systemd[1]: Started exabgp.service - ExaBGP BGP Route Server.
Mar 17 02:41:45 bgp-ser exabgp[18772]: welcome      Thank you for using ExaBGP
Mar 17 02:41:45 bgp-ser exabgp[18772]: cli control   named pipes for the cli are:
Mar 17 02:41:45 bgp-ser exabgp[18772]: cli control   to send commands /run/exabgp/exabgp.in
Mar 17 02:41:45 bgp-ser exabgp[18772]: cli control   to read responses /run/exabgp/exabgp.out
lab@bgp-ser:/etc$ sudo vi /etc/exabgp/exabgp.conf
```

5. Now check your bgp neighborships on your other devices:

```
Cisco_Router# sh ip bgp summ
BGP router identifier 3.3.3.3, local AS number 300
BGP table version is 1, main routing table version 1

Neighbor      V      AS MsgRcvd MsgSent  TblVer  InQ OutQ Up/Down  State/PfxRcd
10.25.1.1     4      200   7363   8107      1    0    0 5d02h    0
10.35.1.1     4      400     3      2      1    0    0 00:00:14  0
Cisco_Router#Connection closed by foreign host.
```

Bring down the exabgp service and check:

```
lab@bgp-ser:/etc$ sudo vi /etc/systemd/system/exabgp.service
lab@bgp-ser:/etc$ sudo systemctl stop exabgp
lab@bgp-ser:/etc$ sudo systemctl start exabgp
```

```
Cisco_Router#sh ip bgp summary
BGP router identifier 3.3.3.3, local AS number 300
BGP table version is 1, main routing table version 1

Neighbor      V      AS MsgRcvd MsgSent   TblVer   InQ OutQ Up/Down  State/PfxRcd
10.25.1.1      4      200    7381    8127       1     0     0 5d02h      0
10.35.1.1      4      400         0         0       1     0     0 00:00:18 Idle
Cisco_Router#
```

Bring it up again:

```
lab@bgp-ser:/etc$ sudo systemctl start exabgp
Cisco_Router#sh ip bgp summary
BGP router identifier 3.3.3.3, local AS number 300
BGP table version is 1, main routing table version 1

Neighbor      V      AS MsgRcvd MsgSent   TblVer   InQ OutQ Up/Down  State/PfxRcd
10.25.1.1      4      200    7382    8127       1     0     0 5d03h      0
10.35.1.1      4      400         3         2       1     0     0 00:00:07      0
Cisco_Router#
```

Section 3 – Establish BGP Peering between Open-Source BGP speaker and SDN Controller AND between Cisco Router and SDN Controller

In this objective, the SDN controller will need to dynamically add flow table entries (“routes”) to the Mininet OvS based on the peers. Failover should be accounted for (i.e. if the Open-Source speaker peering sessions goes down, all traffic should be routed through the other BGP peer).

Setting the IP on the eno1 interface which will be used to establish BGP peering from OVS2 from Server 4 which hosts the Ryu controller -

```
(ryu-venv) lab@sdncont:~$ sudo ip addr add 10.20.1.1/24 dev eno1
(ryu-venv) lab@sdncont:~$ ping 10.20.1.2
PING 10.20.1.2 (10.20.1.2) 56(84) bytes of data:
64 bytes from 10.20.1.2: icmp_seq=1 ttl=255 time=0.552 ms
64 bytes from 10.20.1.2: icmp_seq=2 ttl=255 time=0.498 ms
^C
--- 10.20.1.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1053ms
rtt min/avg/max/mdev = 0.498/0.525/0.552/0.027 ms
(ryu-venv) lab@sdncont:~$ ip add show eno1
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 14:18:77:3b:9f:26 brd ff:ff:ff:ff:ff:ff
    altname enp2s0f0
    inet 10.20.1.1/24 scope global eno1
        valid_lft forever preferred_lft forever
    inet6 fe80::1618:77ff:fe3b:9f26/64 scope link
        valid_lft forever preferred_lft forever
```

Setting the IP on the eno3 to allow for BGP peering with Server3 to Server 4 via OVS1


```
(ryu-venv) lab@sdncont:~$ sudo ip addr add 10.45.1.1/24 dev eno3
(ryu-venv) lab@sdncont:~$ ip addr show eno3
4: eno3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 14:18:77:3b:9f:28 brd ff:ff:ff:ff:ff:ff
    altname enp3s0f0
    inet 10.45.1.1/24 scope global eno3
        valid_lft forever preferred_lft forever
    inet6 fe80::1618:77ff:fe3b:9f28/64 scope link
        valid_lft forever preferred_lft forever
(ryu-venv) lab@sdncont:~$ ping 10.45.1.2
PING 10.45.1.2 (10.45.1.2) 56(84) bytes of data.
64 bytes from 10.45.1.2: icmp_seq=1 ttl=64 time=0.456 ms
64 bytes from 10.45.1.2: icmp_seq=2 ttl=64 time=0.216 ms
64 bytes from 10.45.1.2: icmp_seq=3 ttl=64 time=0.228 ms
^C
--- 10.45.1.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2081ms
rtt min/avg/max/mdev = 0.216/0.300/0.456/0.110 ms
```

The script “sdn_bgp_controller.py” created an BGP peering with the cisco router as well as the Server3 -

```
(ryu-venv) lab@sdncont:~$ sudo /home/lab/ryu-venv/bin/ryu-manager sdn_bgp_controller.py
1 RLock(s) were not greened, to fix this error make sure you run eventlet.monkey_patch() before i
loading app sdn_bgp_controller.py
loading app ryu.controller.ofp_handler
instantiating app sdn_bgp_controller.py of Lab8BGPController
API method core.start called with args: {'waiter': <ryu.lib.hub.Event object at 0x74ad34713c90>,
efresh_max_eor_time': 0, 'label_range': (100, 100000), 'allow_local_as_in_count': 0, 'cluster_id'
API method neighbor.create called with args: {'ip_address': '10.20.1.2', 'remote_as': 300, 'remot
self': False, 'connect_mode': 'both', 'cap_enhanced_refresh': False, 'cap_four_octet_as_number':
ipv4fs': False, 'cap_mbgp_ipv6fs': False, 'cap_mbgp_vpnv4fs': False, 'cap_mbgp_vpnv6fs': False, '
API method neighbor.create called with args: {'ip_address': '10.45.1.2', 'remote_as': 400, 'remot
self': False, 'connect_mode': 'both', 'cap_enhanced_refresh': False, 'cap_four_octet_as_number':
ipv4fs': False, 'cap_mbgp_ipv6fs': False, 'cap_mbgp_vpnv4fs': False, 'cap_mbgp_vpnv6fs': False, '
Ryu (AS 500) Active: Peering with Cisco (300) and OS-Speaker (400)
instantiating app ryu.controller.ofp_handler of OFPHandler
Connection to peer: 10.20.1.2 established
Switch 1 (ovs1) is now CONNECTED!
Switch 2 (ovs2) is now CONNECTED!
Will try to reconnect to 10.45.1.2 after 30 secs: True
Connection to peer: 10.45.1.2 established
```

Cisco router shows the peering UP -

```
Cisco_Router#show ip bgp summary
BGP router identifier 3.3.3.3, local AS number 300
BGP table version is 13, main routing table version 13

Neighbor      V      AS MsgRcvd MsgSent  TblVer  InQ OutQ Up/Down  State/PfxRcd
10.20.1.1     4       500    26     27     13    0   0 00:05:14    0
10.25.1.1     4       200   16059   17686    13    0   0 1w4d        0
10.35.1.1     4       400    32     35     13    0   0 00:29:43    0
Cisco_Router#
```

We could see the BGP route advertised from SDN controller RYU to the Cisco and EXABGP instance 200.X range belongs to the mininet host -

```
self.speaker.prefix_add(prefix='200.1.1.0/24')
```

```

Cisco_Router#show ip route bgp
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       + - replicated route, % - next hop override

Gateway of last resort is not set

B       200.1.1.0/24 [20/0] via 10.20.1.1, 00:03:20
Cisco_Router#

```

We can see when a route is advertised by the peer – it triggers the route programming to use ovs1 or 2 -

```

[SYSTEM] BGP Speaker initialized. Re-Sync logic active.
instantiating app ryu.controller.ofp_handler of OFPHandler
[DISCOVERY] SW3 (Access) ID: 3
Will try to reconnect to 10.45.1.2 after 30 secs: True
Connection to peer: 10.20.1.2 established

=====
      SDN BGP ROUTING TABLE (RIB) UPDATE
      Active Switches: 1
=====
Network Prefix      | Next Hop Address
-----
1.1.1.0/24          | 10.20.1.2
-----
[ORCHESTRATION] BGP Update: 1.1.1.0/24 via 10.20.1.2
-> Rule Pushed to SW3: 1.1.1.0/24 out via sw3-sw1
=====

Connection to peer: 10.45.1.2 established
[DISCOVERY] Edge Switch ID: 22095312166694
-> Rule Pushed to Edge 22095312166694: 1.1.1.0/24 out via eno1
[DISCOVERY] Edge Switch ID: 22095312166696
-> Rule Pushed to Edge 22095312166696: 1.1.1.0/24 out via eno3

```

Section 4 - IP Connectivity from VM to SDN: [🔗](#)

The objective is to achieve communication between a Mininet host (Server 4) and an OpenStack instance (Server 1).

1. Objective

The objective of Section 4 was to achieve IP communication between a Mininet host running on Server 4 and an OpenStack instance running on Server 1. The lab deliverable also requires identifying the path/hops involved and accounting for BGP failover behavior.

2. Environment prepared

I first prepared the OpenStack side on Server 1 by switching from Sunbeam to DevStack after Sunbeam did not function correctly in our environment. DevStack was successfully deployed, and the OpenStack control plane was confirmed operational. My notes show that Keystone,

Nova, Neutron, Glance, Placement, Horizon, and the hypervisor were all up, and a test instance was launched successfully.

On Server 4, I prepared the SDN side by setting up Python/Mininet and Ryu, resolving the Python 3.12/distutils-related issue, and confirming that a Ryu controller could connect to Mininet. I then built the OVS/Mininet topology for Section 4, including sw1, sw2, vs2, and host h1, and migrated link IPs from physical interfaces onto the OVS bridges. I also prepared top.py to create h1, create vs2, patch vs2 into sw1 and sw2, and assign a fixed IP address to h1.

Because this was a group lab and the addressing/state on the devices was not initially consistent, I also performed a full interface/IP inventory across Server 1, Server 3, Server 4, and the Cisco router before continuing. This was done to avoid breaking teammate configurations and to identify address conflicts before proceeding. My notes explicitly record that this topology verification step was necessary before continuing Section 4.

server 1 & 2 connection 10.12.1.1 and 2

```
lab@nvoserver1-openstack:~$ sudo ip addr add 10.12.1.1/24 dev eno3
```

```
lab@frouting:~$ sudo ip link set eno3 up
lab@frouting:~$ ip -br addr
lo                UNKNOWN    127.0.0.1/8 ::1/128
eno1              UP         10.25.1.1/24 fe80::1618:77ff:fe67:104a/64
eno2             DOWN
eno3             UP         10.12.1.2/24 fe80::1618:77ff:fe67:104c/64
eno4             UP         10.0.0.12/24 fe80::1618:77ff:fe67:104d/64
lab@frouting:~$ ip route
default via 10.0.0.1 dev eno4
10.0.0.0/24 dev eno4 proto kernel scope link src 10.0.0.12
10.12.1.0/24 dev eno3 proto kernel scope link src 10.12.1.2
10.25.1.0/24 dev eno1 proto kernel scope link src 10.25.1.1
lab@frouting:~$ ping -c 3 10.12.1.1
PING 10.12.1.1 (10.12.1.1) 56(84) bytes of data:
64 bytes from 10.12.1.1: icmp_seq=1 ttl=64 time=0.364 ms
64 bytes from 10.12.1.1: icmp_seq=2 ttl=64 time=0.189 ms
64 bytes from 10.12.1.1: icmp_seq=3 ttl=64 time=0.193 ms

--- 10.12.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2070ms
rtt min/avg/max/mdev = 0.189/0.248/0.364/0.081 ms
lab@frouting:~$
```

```
lab@frouting:~$ ping -c 3 10.12.1.1
PING 10.12.1.1 (10.12.1.1) 56(84) bytes of data:
64 bytes from 10.12.1.1: icmp_seq=1 ttl=64 time=0.364 ms
64 bytes from 10.12.1.1: icmp_seq=2 ttl=64 time=0.189 ms
64 bytes from 10.12.1.1: icmp_seq=3 ttl=64 time=0.193 ms

--- 10.12.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2070ms
rtt min/avg/max/mdev = 0.189/0.248/0.364/0.081 ms
lab@frouting:~$
```

fixed server 1 dns

```
lab@nvoserver1-openstack:~$ sudo resolvectl dns eno4 8.8.8.8 1.1.1.1
lab@nvoserver1-openstack:~$ sudo resolvectl domain eno4 -.
lab@nvoserver1-openstack:~$ resolvectl status
Global
    Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    resolv.conf mode: stub

Link 2 (eno1)
    Current Scopes: none
    Protocols: -DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported

Link 3 (eno2)
    Current Scopes: none
    Protocols: -DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported

Link 4 (eno3)
    Current Scopes: none
    Protocols: -DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported

Link 5 (eno4)
    Current Scopes: DNS
    Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    Current DNS Server: 8.8.8.8
    DNS Servers: 8.8.8.8 1.1.1.1
    DNS Domain: -.
lab@nvoserver1-openstack:~$ ping -c 2 snapchat.io
PING snapchat.io (185.125.190.21) 56(84) bytes of data:
64 bytes from website-content-cache-2.ps5.canonical.com (185.125.190.21): icmp_seq=1 ttl=54 time=110 m
s
64 bytes from website-content-cache-2.ps5.canonical.com (185.125.190.21): icmp_seq=2 ttl=54 time=110 m
s

--- snapchat.io ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 109.870/109.956/110.042/0.086 ms
lab@nvoserver1-openstack:~$
```

installing openstack on server 1

```
lab@nvoserver1-openstack:~$ sudo snap install openstack
2026-03-12T02:59:43Z INFO Waiting for automatic snapd restart...
openstack (2024.1/stable) 2024.1 from Canonical✓ installed
lab@nvoserver1-openstack:~$ sunbeam --help
Usage: sunbeam [OPTIONS] COMMAND [ARGS]...

Sunbeam is a small lightweight OpenStack distribution.

To get started with a single node, all-in-one OpenStack installation, start
with by initializing the local node. Once the local node has been
initialized, run the bootstrap process to get a live cloud.

Options:
  -q, --quiet
  -v, --verbose
  -h, --help      Show this message and exit.

Commands:
  cloud-config      Generate or Update clouds.yaml.
  cluster           Manage the Sunbeam Cluster.
  configure         Configure cloud with some sensible defaults.
  dashboard-url     Retrieve OpenStack Dashboard URL.
  deployment        Manage deployments.
  disable           Disable features.
  enable            Enable features.
  identity          Manage identity settings.
  juju              Utilities for managing juju.
  launch            Launch an OpenStack instance on demo setup.
  list-feature-gates List all feature gates and their status.
  list-features     List all features and show whether each is enabled...
  manifest          Manage manifests (read-only commands).
  openrc            Retrieve openrc for cloud admin account.
  plans             Manage terraform plans.
  prepare-node-script Generate script to prepare a node for Sunbeam use.
  proxy             Manage proxy configuration.
  storage           Manage Cinder storage backends.
  utils             Utilities for debugging and managing sunbeam.
```



```

lab@nvoserver1-openstack:~$ openstack --help
usage: openstack [--version] [-v | -q] [--log-file LOG_FILE] [-h] [--debug]
               [--os-cloud <cloud-config-name>]
               [--os-region-name <auth-region-name>]
               [--os-cacert <ca-bundle-file>] [--os-cert <certificate-file>]
               [--os-key <key-file>] [--verify | --insecure]
               [--os-default-domain <auth-domain>] [--os-interface <interface>]
               [--os-service-provider <service_provider>]
               [--os-remote-project-name <remote_project_name> | --os-remote-project-id <remote_proje
ct_id>]
               [--os-remote-project-domain-name <remote_project_domain_name> | --os-remote-project-do
main-id <remote_project_domain_id>]
               [--timing] [--os-beta-command] [--os-profile hmac-key]
               [--os-compute-api-version <compute-api-version>]
               [--os-identity-api-version <identity-api-version>]
               [--os-image-api-version <image-api-version>]
               [--os-network-api-version <network-api-version>]
               [--os-object-api-version <object-api-version>]
               [--os-volume-api-version <volume-api-version>]
               [--os-alarming-api-version <alarming-api-version>]
               [--os-metrics-api-version <metrics-api-version>]
               [--os-placement-api-version <placement-api-version>]
               [--os-key-manager-api-version <key-manager-api-version>]
               [--rating-api-version RATING_API_VERSION]
               [--os-policy-api-version <policy-api-version>]
               [--os-dns-api-version <dns-api-version>]
               [--os-orchestration-api-version <orchestration-api-version>]
               [--os-baremetal-api-version <baremetal-api-version>]
               [--os-container-infra-api-version <container-infra-api-version>]
               [--os-share-api-version <shared-file-system-api-version>]
               [--os-ha-api-version <ha-api-version>]
               [--os-workflow-api-version <workflow-api-version>]
               [--os-application-catalog-api-version <application-catalog-api-version>]
               [--murano-url MURANO_URL] [--glare-url GLARE_URL]
               [--murano-packages-service {murano,glare}]
               [--os-loadbalancer-api-version <loadbalancer-api-version>]
               [--os-data-processing-api-version <data-processing-api-version>]
               [--os-data-processing-url OS_DATA_PROCESSING_URL]
               [--os-clustering-api-version <clustering-api-version>]
               [--os-tacker-api-version <tacker-api-version>]
               [--os-database-api-version <database-api-version>]
               [--os-rca-api-version <rca-api-version>]
               [--os-infra-optim-api-version <infra-optim-api-version>]
               [--os-queues-api-version <queues-api-version>]
               [--os-auth-type <auth-type>] [--os-auth-url <auth-auth-url>]
               [--os-system-scope <auth-system-scope>]
               [--os-domain-id <auth-domain-id>]
               [--os-domain-name <auth-domain-name>]
               [--os-project-id <auth-project-id>]
               [--os-project-name <auth-project-name>]

```

```

lab@nvoserver1-openstack:~$ snap info openstack
name:      openstack
summary:   Your own enterprise-grade cloud platform
publisher: Canonical✓
store-url: https://snapcraft.io/openstack
contact:   https://matrix.to/#/#openstack-sunbeam:ubuntu.com
license:   Apache-2.0
description: |
  This snap installs Sunbeam - a modern framework for automating OpenStack deployment and
  operations processes. Build your cloud in a few simple steps and operate it with ease at any
  scale.

```

```

lab@nvoserver1-openstack:~$ sunbeam prepare-node-script --bootstrap > bootstrap.sh
lab@nvoserver1-openstack:~$ head -40 bootstrap.sh
#!/bin/bash
[ $(lsb_release -sc) != 'noble' ] && { echo 'ERROR: Sunbeam deploy only supported on noble'; exit 1; }

# ▲Node Preparation for OpenStack Sunbeam ▲
# All of these commands perform privileged operations
# please review carefully before execution.
USER=$(whoami)

if [ $(id -u) -eq 0 -o "$USER" = root ]; then
    cat << EOF
ERROR: Node Preparation script for OpenStack Sunbeam must be executed by
    non-root user with sudo permissions.
EOF
    exit 1
fi

# Check if user has passwordless sudo permissions and setup if need be
SUDO_ASKPASS=/bin/false sudo -A whoami && /dev/null &&
sudo grep -r $USER /etc/{sudoers,sudoers.d} | grep NOPASSWD:ALL && /dev/null || {
    echo "$USER ALL=(ALL) NOPASSWD:ALL" > /tmp/90-$USER-sudo-access
    sudo install -m 440 /tmp/90-$USER-sudo-access /etc/sudoers.d/90-$USER-sudo-access
    rm -f /tmp/90-$USER-sudo-access
}

# Ensure parallel instances of snaps is enabled
if [ "$(sudo snap get system experimental.parallel-instances 2>/dev/null)" != "true" ];
then
    sudo snap set system experimental.parallel-instances=true
    # Force recreate the snap sandbox LP#2139363
    sudo /usr/lib/snapd/snap-discard-ns openstack
    sudo snap restart openstack
fi

# Ensure dependency packages are installed
for pkg in openssh-server curl sed; do
    dpkg -s $pkg &> /dev/null || {
        sudo apt install -y $pkg
    }
done

Now you can run
    juju add-model <model-name>
to create a new model to deploy workloads.
+ echo 'Juju bootstrap complete, you can now bootstrap sunbeam!'
Juju bootstrap complete, you can now bootstrap sunbeam!
lab@nvoserver1-openstack:~$ newgrp snap_daemon
lab@nvoserver1-openstack:~$ id
uid=1000(lab) gid=584788(snap_daemon) groups=584788(snap_daemon),4(adm),24(cdrom),27(sudo),30(dip),46(p
lugdev),101(lxd),1000(lab)
lab@nvoserver1-openstack:~$ groups
snap_daemon adm cdrom sudo dip plugdev lxd lab
lab@nvoserver1-openstack:~$

```

Sunbeam seems not functioning so switching to devstack

```
lab@nvoserver1-openstack:~$ sudo apt install -y git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
lab@nvoserver1-openstack:~$ sudo useradd -s /bin/bash -d /opt/stack -m stack
lab@nvoserver1-openstack:~$ echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
stack ALL=(ALL) NOPASSWD: ALL
lab@nvoserver1-openstack:~$ sudo -iu stack
stack@nvoserver1-openstack:~$ git clone https://opendev.org/openstack/devstack
Cloning into 'devstack'...
remote: Enumerating objects: 52482, done.
remote: Counting objects: 100% (32022/32022), done.
remote: Compressing objects: 100% (10968/10968), done.
remote: Total 52482 (delta 31241), reused 21054 (delta 21054), pack-reused 20460 (from 1)
Receiving objects: 100% (52482/52482), 9.85 MiB | 12.34 MiB/s, done.
Resolving deltas: 100% (37265/37265), done.
stack@nvoserver1-openstack:~$ cd devstack
stack@nvoserver1-openstack:~/devstack$
```

./unstack.sh

./clean.sh

./stack.sh

devstack setup!!!


```
lab@nvoserver1-openstack: /c  +  -  x
| nova_cell0 | SHOW      | 59 |
| nova_cell0 | SELECT    | 81 |
| nova_cell1 | ALTER     | 15 |
| nova_cell1 | SHOW      | 60 |
| nova_cell1 | SELECT    | 154 |
| nova_cell0 | INSERT    | 11 |
| nova_cell1 | INSERT    | 6 |
| nova_cell0 | DROP      | 2 |
| nova_cell1 | DROP      | 4 |
| nova_cell0 | UPDATE    | 20 |
| nova_cell1 | UPDATE    | 82 |
| nova_api   | SELECT    | 63 |
| placement | SELECT    | 28 |
| placement | INSERT    | 73 |
| placement | SET       | 3 |
| cinder     | SELECT    | 26 |
| cinder     | INSERT    | 2 |
| placement | UPDATE    | 3 |
| nova_api   | SAVEPOINT | 12 |
| nova_api   | RELEASE   | 12 |
+-----+-----+-----+

This is your host IP address: 10.0.0.11
This is your host IPv6 address: ::1
Horizon is now available at http://10.0.0.11/dashboard
Keystone is serving at http://10.0.0.11/identity/
The default users are: admin and demo
The password: labstack

Services are running under systemd unit files.
For more information see:
https://docs.openstack.org/devstack/latest/systemd.html

DevStack Version: 2026.1
Change: 416d27e89e0c1891921fee2a692086eb8fcd0307 Add the config file referenc
e argument to ``configure_keystoneauth`` 2026-03-11 17:05:13 +0100
OS Version: Ubuntu 24.04 noble

stack@nvoserver1-openstack:~/devstack$
```

backup :

cd /opt/stack/devstack

cp local.conf ~/local.conf.backup

source openrc admin admin

openstack service list > ~/os_service_list.txt

openstack network list > ~/os_network_list.txt

openstack subnet list > ~/os_subnet_list.txt

openstack router list > ~/os_router_list.txt

openstack hypervisor list > ~/os_hypervisor_list.txt

openstack flavor list > ~/os_flavor_list.txt

openstack image list > ~/os_image_list.txt

ip a > ~/ip_a_before_reboot.txt

ip route > ~/ip_route_before_reboot.txt

```
cat /etc/netplan/*.yaml > ~/netplan_backup.txt
```

conclusion:

DevStack on Server 1 is successfully deployed and operational. Keystone, Nova, Neutron, Glance, Placement, Horizon, and the hypervisor are up. Default networks, subnets, router, flavors, and CirrOS images are present. A test instance was successfully launched and reached ACTIVE state with an assigned private IP, confirming that the core OpenStack control plane and instance provisioning path are functioning.

Server 4:

setting up mininet/pip3

```
lab@sdncont:~$ which ovs-vsctl
/usr/bin/ovs-vsctl
lab@sdncont:~$ which mn
/usr/bin/mn
lab@sdncont:~$ pip3 --version
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
lab@sdncont:~$
```

had trouble with python 3.12 removing distutils

so had to <https://askubuntu.com/questions/1239829/moduleNotFoundError-no-module-named-distutils-util>

The screenshot shows a forum post with a score of 221. The user describes a problem after upgrading to Ubuntu 20.04, where a virtual environment depending on Python 3.7 failed due to a missing `distutils` module. The solution involves adding the `deadsnake PPA` and installing `python3.7-distutils`. A note mentions an additional error was solved by forcing the installation of the package.

I got this problem after upgrading to Ubuntu 20.04. I had a virtual environment depending on Python 3.7, and, to avoid re-installing the whole virtual environment for Python 3.8, I fixed `distutils` on Python 3.7:

I added the `deadsnake PPA`:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt-get update
```

And then installed the `distutils` package for Python 3.7:

```
sudo apt install python3.7-distutils
```

Note: for some reason I had an error installing the latter, that I solved [this way](#):

```
sudo dpkg -i --force-overwrite /var/cache/apt/archives/python3
sudo apt-get -f install
```

able to connect to mininet

```

lab@sdncont:~$ ss -ltnp | egrep '6633|6653|8080'
LISTEN 0      50          0.0.0.0:6653  0.0.0.0:*    users:(("ryu-manager",pid=30833,fd=5))
LISTEN 0      50          0.0.0.0:6633  0.0.0.0:*    users:(("ryu-manager",pid=30833,fd=4))
lab@sdncont:~$ sudo mn --topo single,3 --mac --switch ovsk --controller remote
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet>

```

```

(ryu-venv) lab@sdncont:~$ ryu-manager ryu.app.simple_switch_13
1 RLock(s) were not greened, to fix this error make sure you run eventlet.mo
nkey_patch() before importing any other modules.
loading app ryu.app.simple_switch_13
loading app ryu.controller.ofp_handler
instantiating app ryu.app.simple_switch_13 of SimpleSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
packet in 1 00:00:00:00:00:01 33:33:00:00:00:16 1
packet in 1 00:00:00:00:00:03 33:33:00:00:00:16 3
packet in 1 00:00:00:00:00:01 33:33:ff:00:00:01 1
packet in 1 00:00:00:00:00:03 33:33:ff:00:00:03 3
packet in 1 00:00:00:00:00:02 33:33:00:00:00:16 2
packet in 1 00:00:00:00:00:02 33:33:00:00:00:16 2
packet in 1 00:00:00:00:00:02 33:33:00:00:00:02 2
packet in 1 00:00:00:00:00:02 33:33:00:00:00:16 2
packet in 1 00:00:00:00:00:01 33:33:00:00:00:16 1
packet in 1 00:00:00:00:00:01 33:33:00:00:00:02 1
packet in 1 00:00:00:00:00:01 33:33:00:00:00:16 1
packet in 1 00:00:00:00:00:03 33:33:00:00:00:16 3
packet in 1 00:00:00:00:00:03 33:33:00:00:00:02 3
packet in 1 00:00:00:00:00:03 33:33:00:00:00:16 3
packet in 1 00:00:00:00:00:01 ff:ff:ff:ff:ff:ff 1
packet in 1 00:00:00:00:00:02 00:00:00:00:00:01 2
packet in 1 00:00:00:00:00:01 00:00:00:00:00:02 1
packet in 1 00:00:00:00:00:01 ff:ff:ff:ff:ff:ff 1
packet in 1 00:00:00:00:00:03 00:00:00:00:00:01 3
packet in 1 00:00:00:00:00:01 00:00:00:00:00:03 1
packet in 1 00:00:00:00:00:02 ff:ff:ff:ff:ff:ff 2
packet in 1 00:00:00:00:00:03 00:00:00:00:00:02 3
packet in 1 00:00:00:00:00:02 00:00:00:00:00:03 2
packet in 1 00:00:00:00:00:02 33:33:00:00:00:02 2
packet in 1 00:00:00:00:00:03 33:33:00:00:00:02 3
packet in 1 00:00:00:00:00:01 33:33:00:00:00:02 1
packet in 1 00:00:00:00:00:02 33:33:00:00:00:02 2
packet in 1 00:00:00:00:00:01 33:33:00:00:00:02 1
packet in 1 00:00:00:00:00:03 33:33:00:00:00:02 3
packet in 1 00:00:00:00:00:02 33:33:00:00:00:02 2
packet in 1 00:00:00:00:00:01 33:33:00:00:00:02 1
packet in 1 00:00:00:00:00:03 33:33:00:00:00:02 3

```

3. Validation Performed

After preparing the OpenStack side on Server 1 and the SDN/Mininet side on Server 4, I validated the Section 4 path as far as possible in the shared group environment.

On the control-plane side, I successfully brought up the required BGP peerings on Server 4 for path validation. The Cisco Router ↔ Server 4 peering and the open-source BGP speaker / Server 3 ↔ Server 4 peering were both established during testing. This satisfied the required control-plane preparation needed before Section 4 connectivity testing.

On the data-plane side, I confirmed that the OpenStack instance test-vm was active and assigned a floating IP (172.24.4.242). I then generated ICMP traffic from the Mininet host h1 (4.4.4.4) toward the floating IP. Packet captures on Server 1 showed that the ICMP echo requests from 4.4.4.4 successfully reached the OpenStack side, and that the floating IP endpoint generated ICMP echo replies back toward 4.4.4.4.

This means that I successfully validated:

- the OpenStack VM and floating IP setup,
- the Server 4 Mininet/OVS topology,
- the required Section 3 control-plane setup needed for Section 4,
- and the forward path from the Mininet host to the OpenStack side, including reply generation from the OpenStack floating IP.

Connection between server 4 and cisco router

```

(ryu-venv) lab@sdncont:~$ sudo ip addr add 10.45.1.1/24 dev eno1
(ryu-venv) lab@sdncont:~$ sudo ip link set eno1 up
(ryu-venv) lab@sdncont:~$
(ryu-venv) lab@sdncont:~$ ip addr show eno1
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group
default qlen 1000
    link/ether 14:18:77:3b:9f:26 brd ff:ff:ff:ff:ff:ff
    altname enp2s0f0
    inet 10.45.1.1/24 scope global eno1
        valid_lft forever preferred_lft forever
    inet6 fe80::1618:77ff:fe3b:9f26/64 scope link
        valid_lft forever preferred_lft forever
(ryu-venv) lab@sdncont:~$ ping -c 3 10.45.1.2
PING 10.45.1.2 (10.45.1.2) 56(84) bytes of data.
64 bytes from 10.45.1.2: icmp_seq=1 ttl=255 time=1.67 ms
64 bytes from 10.45.1.2: icmp_seq=2 ttl=255 time=0.552 ms
64 bytes from 10.45.1.2: icmp_seq=3 ttl=255 time=0.434 ms

--- 10.45.1.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2020ms
rtt min/avg/max/mdev = 0.434/0.884/1.668/0.555 ms
(ryu-venv) lab@sdncont:~$

```

```

Cisco_Router#show ip int br
Interface                               IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0                     10.0.0.15       YES manual up                    up
GigabitEthernet0/1                     unassigned      YES unset administratively down down
Serial0/0/0                             unassigned      YES unset administratively down down
Serial0/0/1                             unassigned      YES unset administratively down down
GigabitEthernet0/1/0                   unassigned      YES unset administratively down down
Serial0/2/0                             unassigned      YES unset administratively down down
Serial0/2/1                             unassigned      YES unset administratively down down
GigabitEthernet0/3/0                   unassigned      YES unset administratively down down
FastEthernet1/0                         10.25.1.2       YES manual up                    up
FastEthernet1/1                         10.35.1.2       YES manual up                    up
FastEthernet2/0                         unassigned      YES unset up                        up
FastEthernet2/1                         unassigned      YES unset administratively down down
Cisco_Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Cisco_Router(config)#int fa2/0
Cisco_Router(config-if)#ip addr 10.45.1.2 255.255.255.0
Cisco_Router(config-if)#end
Cisco_Router#wr

```

Then server 4 ovs/mininet


```

(ryu-venv) lab@sdncont:~$ sudo ovs-vsctl --if-exists del-br sw1
(ryu-venv) lab@sdncont:~$ sudo ovs-vsctl --if-exists del-br sw2
(ryu-venv) lab@sdncont:~$ sudo ovs-vsctl show
abfcfef0-a47b-4f24-b643-0d919f8ce218
    ovs_version: "3.3.4"
(ryu-venv) lab@sdncont:~$ sudo ovs-vsctl add-br sw1
(ryu-venv) lab@sdncont:~$ sudo ovs-vsctl add-br sw2
(ryu-venv) lab@sdncont:~$ sudo ovs-vsctl show
abfcfef0-a47b-4f24-b643-0d919f8ce218
    Bridge sw1
        Port sw1
            Interface sw1
                type: internal
    Bridge sw2
        Port sw2
            Interface sw2
                type: internal
    ovs_version: "3.3.4"

```

bring link from physical ip to bridge ip


```

(ryu-venv) lab@sdncont:~$ sudo ip addr del 10.20.1.4/24 dev eno3
(ryu-venv) lab@sdncont:~$ sudo ip addr del 10.45.1.1/24 dev eno1
(ryu-venv) lab@sdncont:~$ sudo ip addr add 10.20.1.4/24 dev sw1
(ryu-venv) lab@sdncont:~$ sudo ip addr add 10.45.1.1/24 dev sw2
(ryu-venv) lab@sdncont:~$ sudo ip link set sw1 up
(ryu-venv) lab@sdncont:~$ sudo ip link set sw2 up
(ryu-venv) lab@sdncont:~$ sudo ovs-vsctl add-port sw1 eno3
(ryu-venv) lab@sdncont:~$ sudo ovs-vsctl add-port sw2 eno1
(ryu-venv) lab@sdncont:~$ ip addr show sw1
34: sw1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKN
OWN group default qlen 1000
    link/ether 14:18:77:3b:9f:28 brd ff:ff:ff:ff:ff:ff
    inet 10.20.1.4/24 scope global sw1
        valid_lft forever preferred_lft forever
    inet6 fe80::88:a7ff:fe45:c942/64 scope link
        valid_lft forever preferred_lft forever
(ryu-venv) lab@sdncont:~$ ip addr show sw2
35: sw2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKN
OWN group default qlen 1000
    link/ether 14:18:77:3b:9f:26 brd ff:ff:ff:ff:ff:ff
    inet 10.45.1.1/24 scope global sw2
        valid_lft forever preferred_lft forever
    inet6 fe80::c07a:58ff:fedd:b74a/64 scope link
        valid_lft forever preferred_lft forever

```

```
(ryu-venv) lab@sdncont:~$ ryu-manager ryu.app.simple_switch_13
1 RLock(s) were not greened, to fix this error make sure you run eventlet.mo
nkey_patch() before importing any other modules.
loading app ryu.app.simple_switch_13
loading app ryu.controller.ofp_handler
instantiating app ryu.app.simple_switch_13 of SimpleSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
packet in 2 14:18:77:3b:9f:26 33:33:00:00:00:02 3
packet in 2 00:00:00:00:00:01 33:33:00:00:00:16 1
packet in 2 00:00:00:00:00:01 33:33:ff:00:00:01 1
packet in 2 00:00:00:00:00:01 33:33:00:00:00:16 1
packet in 2 00:00:00:00:00:01 33:33:00:00:00:02 1
packet in 2 00:00:00:00:00:01 33:33:00:00:00:16 1
packet in 2 00:16:c7:9a:c4:f9 ab:00:00:02:00:00 3
packet in 2 00:00:00:00:00:01 33:33:00:00:00:02 1
packet in 2 00:00:00:00:00:01 33:33:00:00:00:02 1
packet in 2 14:18:77:3b:9f:28 33:33:00:00:00:02 2
packet in 2 14:18:77:3b:9f:26 33:33:00:00:00:02 3
packet in 2 00:00:00:00:00:01 33:33:00:00:00:02 1
packet in 2 00:00:00:00:00:01 33:33:00:00:00:02 1
packet in 2 14:18:77:3b:9f:26 33:33:00:00:00:02 3
packet in 2 14:18:77:3b:9f:28 33:33:00:00:00:02 2
packet in 2 00:00:00:00:00:01 33:33:00:00:00:02 1
|
```

lab@sdncont: ~

×

+

▼

```
lab@sdncont:~$ sudo ovs-vsctl show
abfcfef0-a47b-4f24-b643-0d919f8ce218
    Bridge sw2
        Port eno1
            Interface eno1
        Port sw2
            Interface sw2
                type: internal
        Port patch-sw2-vs2
            Interface patch-sw2-vs2
                type: patch
                options: {peer=patch-vs2-sw2}
    Bridge sw1
        Port sw1
            Interface sw1
                type: internal
        Port patch-sw1-vs2
            Interface patch-sw1-vs2
                type: patch
                options: {peer=patch-vs2-sw1}
        Port eno3
            Interface eno3
    Bridge vs2
        Controller "tcp:127.0.0.1:6653"
            is_connected: true
        fail_mode: secure
        Port patch-vs2-sw1
            Interface patch-vs2-sw1
                type: patch
                options: {peer=patch-sw1-vs2}
        Port vs2-eth1
            Interface vs2-eth1
        Port patch-vs2-sw2
            Interface patch-vs2-sw2
                type: patch
                options: {peer=patch-sw2-vs2}
        Port vs2
            Interface vs2
                type: internal
```

After running bgp_app.py

```
(ryu-venv) lab@sdncont:~$ sudo -E ~/ryu-venv/bin/python ~/bgp_app.py
[sudo] password for lab:
/home/lab/bgp_app.py:2: DeprecationWarning:
Eventlet is deprecated. It is currently being maintained in bugfix mode, and
we strongly recommend against using it for new projects.

If you are already using Eventlet, we recommend migrating to a different
framework. For more detail see
https://eventlet.readthedocs.io/en/latest/asyncio/migration.html

import eventlet
BGP speaker started on AS500
Neighbors: 10.20.1.3 (AS400), 10.45.1.2 (AS300)
Advertising: 4.4.4.0/24
=== PEER UP: 3.3.3.3 AS300 ===
AS_PATH on UPDATE message has loops. Ignoring this message: BGPUpdate(len=51
,nlri=[BGPNLRI(addr='4.4.4.0',length=24)],path_attributes=[BGPPathAttribute0
rigin(flags=64,length=1,type=1,value=0), BGPPathAttributeAsPath(flags=64,len
gth=10,type=2,value=[[300, 500]]), BGPPathAttributeNextHop(flags=64,length=4
,type=3,value='10.45.1.2')],total_path_attribute_len=24,type=2,withdrawn_rou
tes=[],withdrawn_routes_len=0)
```

We created a separate ExaBGP configuration file, `exabgp_s4.conf`, so we could bring up the **Server 3** ↔ **Server 4** BGP session needed for our Section 4 path validation without modifying the team's original `exabgp.conf`. The original file was already being used for another peering design, so changing it directly could break an existing working setup or interfere with a teammate's configuration. By using a dedicated config file, we isolated our test, reduced risk, made troubleshooting easier, and kept the original configuration intact for rollback or comparison.

We performed a full interface and IP inventory across Server 1, Server 3, Server 4, and the Cisco router before making any additional routing changes. This was necessary because the lab is a team project, and changing IPs or BGP settings without a complete topology check could disrupt an existing teammate configuration. The inventory confirmed that Server 4 is correctly set up with `sw1 = 10.20.1.4/24`, `sw2 = 10.45.1.1/24`, and the OpenStack server is reachable on the management/external side. However, it also revealed an address conflict or topology mismatch: `10.45.1.2` appears both on Server 3 `eno3` and on Cisco `FastEthernet2/0`. Because of that inconsistency, we paused further BGP changes and treated topology verification as the next required step before continuing Section 4.


```

Cisco_Router#show ip bgp su
BGP router identifier 3.3.3.3, local AS number 300
BGP table version is 4, main routing table version 4
1 network entries using 136 bytes of memory
1 path entries using 56 bytes of memory
1/1 BGP path/bestpath attribute entries using 128 bytes of memory
1 BGP AS-PATH entries using 24 bytes of memory
0 BGP route-map cache entries using 0 bytes of memory
0 BGP filter-list cache entries using 0 bytes of memory
BGP using 344 total bytes of memory
BGP activity 2/1 prefixes, 2/1 paths, scan interval 60 secs

Neighbor      V      AS MsgRcvd MsgSent   TblVer  InQ OutQ Up/Down  State/Pf
xRcd
10.20.1.1      4      500      3       4        4    0    0 00:00:07    1
10.25.1.1      4      200    12962   14274        4    0    0 1w2d        0
10.35.1.1      4      400      0       0        1    0    0 00:31:51 Idle

Cisco_Router#ping 10.20.1.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.20.1.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/4 ms
Cisco_Router#show tcp brief | include 179
69FF7028 10.25.1.2.179                10.25.1.1.40750      ESTAB
6A067024 10.20.1.2.179                10.20.1.1.40770      ESTAB
Cisco_Router#

```

```

(ryu-venv) lab@sdncont:~$ sudo -E ~/ryu-venv/bin/python ~/bgp_app.py
/home/lab/bgp_app.py:2: DeprecationWarning:
Eventlet is deprecated. It is currently being maintained in bugfix mode, and
we strongly recommend against using it for new projects.

If you are already using Eventlet, we recommend migrating to a different
framework. For more detail see
https://eventlet.readthedocs.io/en/latest/asyncio/migration.html

import eventlet
BGP speaker started on AS500
Neighbors: 10.45.1.2 (AS400), 10.20.1.2 (AS300)
Advertising: 4.4.4.0/24
=== PEER UP: 3.3.3.3 AS300 ===
AS_PATH on UPDATE message has loops. Ignoring this message: BGPUpdate(len=51
,nlri=[BGPNLRI(addr='4.4.4.0',length=24)],path_attributes=[BGPPathAttributeO
rigin(flags=64,length=1,type=1,value=0), BGPPathAttributeAsPath(flags=64,len
gth=10,type=2,value=[[300, 500]]), BGPPathAttributeNextHop(flags=64,length=4
,type=3,value='10.20.1.2')],total_path_attribute_len=24,type=2,withdrawn_rou
tes=[],withdrawn_routes_len=0)

```

After fixing the ips

```
(ryu-venv) lab@sdncont:~$ sudo -E ~/ryu-venv/bin/python ~/bgp_app.py
/home/lab/bgp_app.py:2: DeprecationWarning:
Eventlet is deprecated. It is currently being maintained in bugfix mode, and
we strongly recommend against using it for new projects.

If you are already using Eventlet, we recommend migrating to a different
framework. For more detail see
https://eventlet.readthedocs.io/en/latest/asyncio/migration.html

import eventlet
BGP speaker started on AS500
Neighbors: 10.45.1.2 (AS400), 10.20.1.2 (AS300)
Advertising: 4.4.4.0/24
=== PEER UP: 3.3.3.3 AS300 ===
AS_PATH on UPDATE message has loops. Ignoring this message: BGPUpdate(len=51
,nlri=[BGPNLRI(addr='4.4.4.0',length=24)],path_attributes=[BGPPathAttribute0
rigin(flags=64,length=1,type=1,value=0), BGPPathAttributeAsPath(flags=64,len
gth=10,type=2,value=[[300, 500]]), BGPPathAttributeNextHop(flags=64,length=4
,type=3,value='10.20.1.2')],total_path_attribute_len=24,type=2,withdrawn_rou
tes=[],withdrawn_routes_len=0)
=== PEER UP: 10.45.1.2 AS400 ===
```

```
lab@bgp-ser:~$
lab@bgp-ser:~$ sudo pkill -f exabgp
lab@bgp-ser:~$ sleep 2
^[[A^[[A^[[A^[[sudo /usr/bin/python3 /usr/local/bin/exabgp server /etc/exabg
p/exabgp.conf
welcome      Thank you for using ExaBGP
cli control  named pipes for the cli are:
cli control  to send commands /run/exabgp/exabgp.in
cli control  to read responses /run/exabgp/exabgp.out
```

Final Deliverable

- Indicate via code that OpenStack host machine (host1) has connectivity to Mininet Host (host2), noting the correct hop/paths along the way.
 - The BGP failover path must be accounted for.

Section 5 – [Extra Credit] BGP via Containers

Implement one of the sections/objectives in a containerized form.

Section 2 has been containerized.

1. Dockerfile used:

```
FROM ubuntu:22.04

RUN apt update && \
    apt install -y \
        python3-pip \
        git \
        vim \
        iproute2 \
        iputils-ping \
        net-tools \
        tcpdump && \
    pip3 install --no-cache-dir exabgp && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /root

CMD ["/bin/bash"]
```

2. Build the exabgp container image
`sudo docker build -t exabgp-image2 .`

3. Run the exabgp container
`sudo docker run -it --network host --name exagp-cont2 exabgp-image2`

```
lab@bgp-ser:~/exabgp_cont$ sudo docker run -it --network host --name exagp-cont2 exabgp-image2
root@bgp-ser:~# ip a
```

4. Now repeat all steps/configs mentioned in Section 2. Since this one is a container, directly run the command `exabgp exabgp.conf` once all configs are made

5. Now verify that BGP peers are being formed in cisco router:

```
Cisco_Router#sh ip bgp summary
BGP router identifier 3.3.3.3, local AS number 300
BGP table version is 13, main routing table version 13

Neighbor      V         AS MsgRcvd MsgSent  TblVer  InQ OutQ Up/Down  State/PfxRcd
10.20.1.1      4          500      0        0        1    0    0 00:05:15 Active
10.25.1.1      4          200    16077    17706    13    0    0 1w4d      0
10.35.1.1      4          400      3        3       13    0    0 00:00:02      0
Cisco Router#|
```

Section 6 – [Extra Credit] BGP via OpenStack

Implement BGP on the OpenStack server.